UBICOMP ISWC2025

14-16 Oct. 2025
Helsinki, Finland

OGC GeoPose

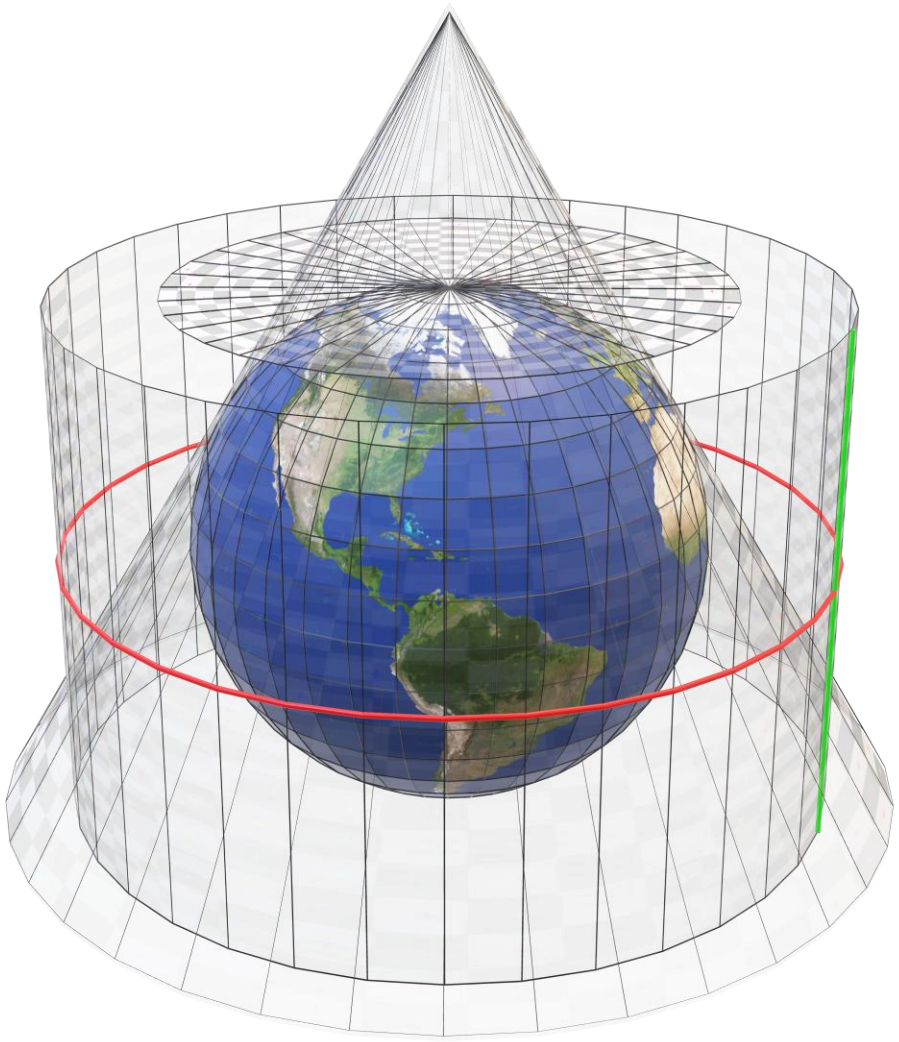
Mikel Salazar, Ph.D.
VR Department. IFE
Halden, Norway

OPEN AR CLOUD

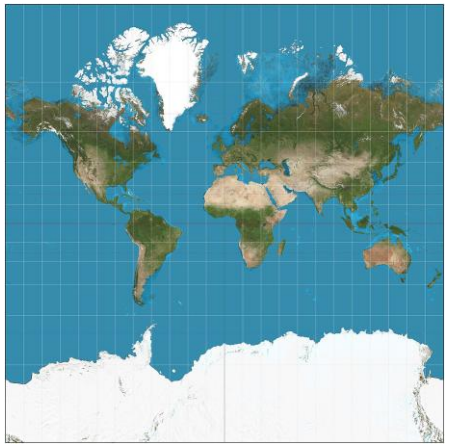
Lack of Interoperability in (Geo)Spatial Computing

Working with Geospatial Information Systems (GIS) it is often difficult, because there are thousands of ways to project the ellipsoidal nature of the Earth on a 2D plane. **Each projection is perfectly valid… but, often, different enough to be incompatible with the rest.**

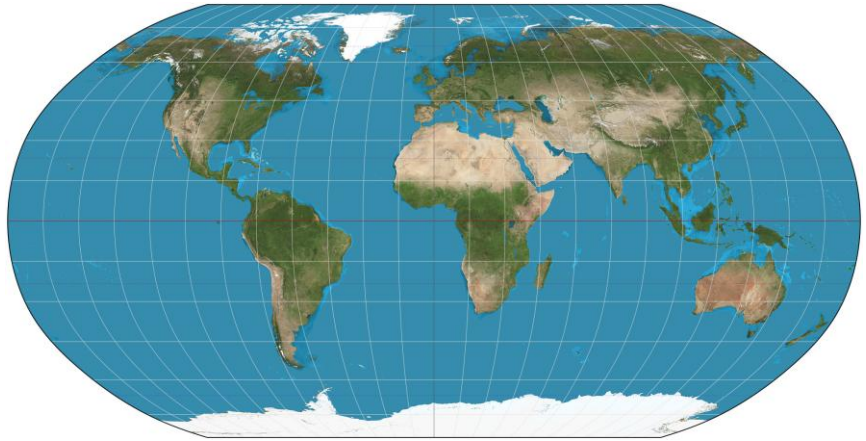
Each geographic projection defines a 2D space based on a Coordinate Reference System (CRS), a reference ellipsoid and an origin point. And, while most projections use degrees of longitude and latitude as units, the dimensions of the space might not be the same, the origin might be linked to a physical marker, and then… there are many other factors to take into consideration like continental drift, earthquakes, geopolitics, etc.



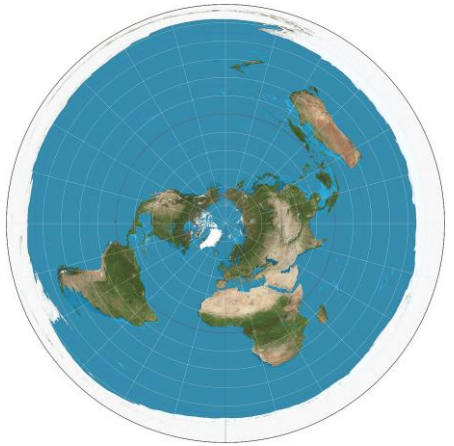
Examples of Geographic Projections (and their spaces and CRSs)



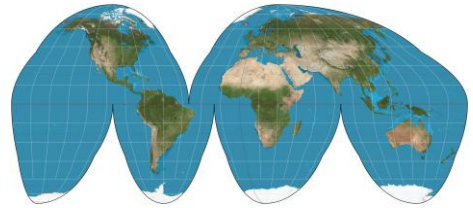
Mercator (EPSG:3832)



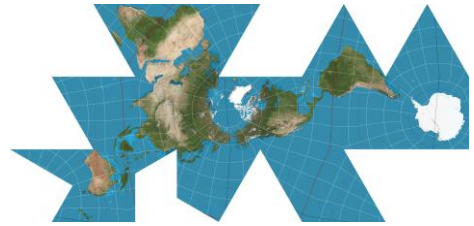
Robinson (ESRI:53030, 54030)



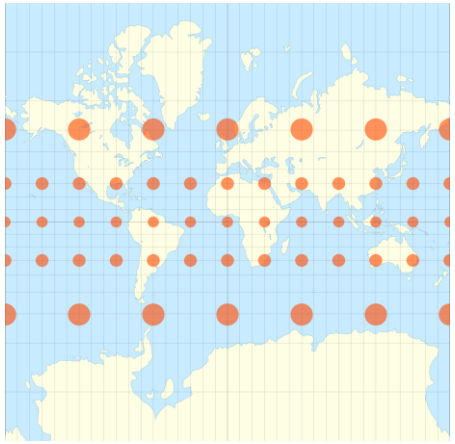
Azimuthal (ESRI:53032)

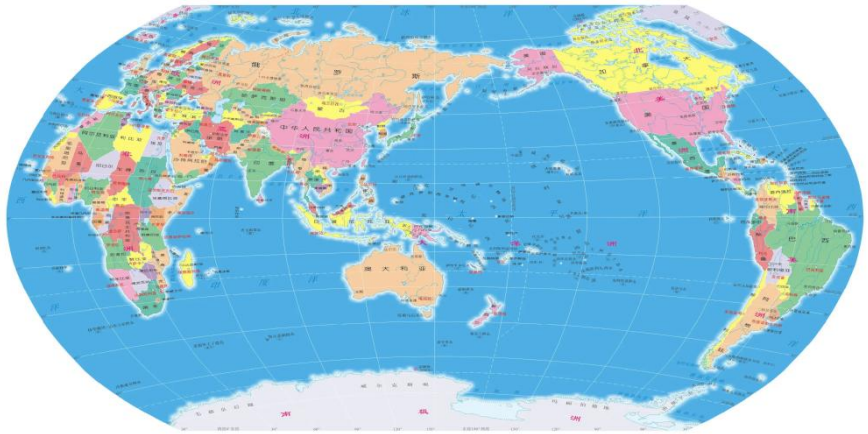


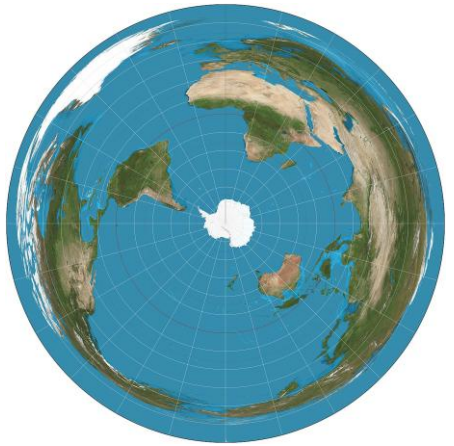
Goode homolosine

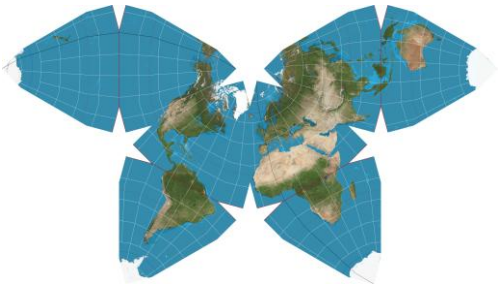


Fuller's Dymaxion









Waterman butterfly

Standardization of Geospatial Data

Fortunately, the Open Geospatial Consortium (OGC) has been collaborating in recent years with organizations as the Khronos Group and the World Wide Web Consortium (W3C) **to standardize geospatial data formats and APIs.**

Notable examples are **the 3D Tiles and GeoTIFF standards**, that not only have greatly simplified the sharing of 3D map data, but also **have become the foundation upon which entire ecosystems have been built.**

Yet, these data formats are mainly focus on static elements. **To be able to track dynamic entities, a new standard was necessary…**

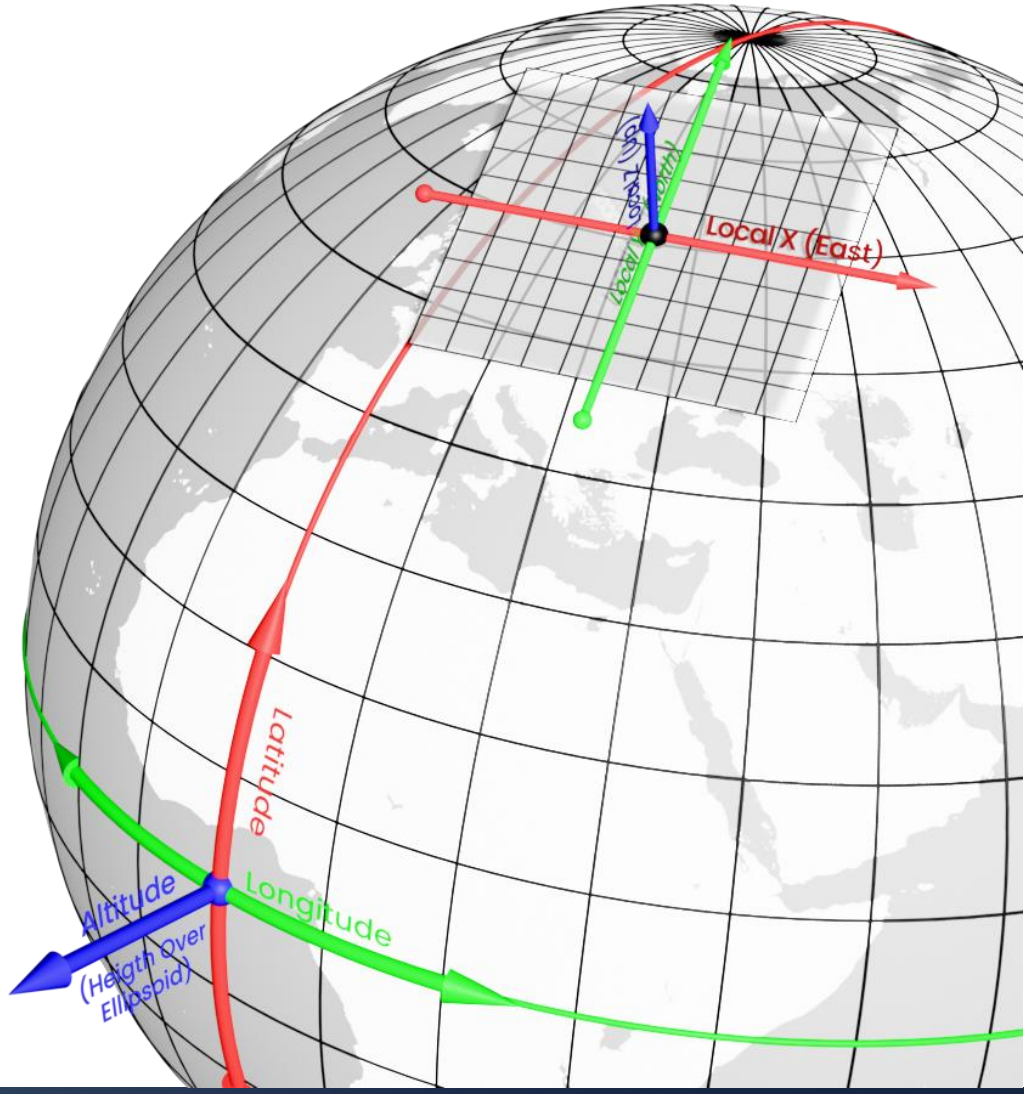


Introducing the OGC GeoPose Standard

The **OGC Geopose Standard defines the encodings for the real world position and orientation of a real or a digital object in a machine-readable (JSON) form .**

In practice, **it provides a simple way to create a transformation matrix based on the World Geodetic System (WGS84) ellipsoid**, to specify the position and rotation of anything relative to the surface of the planet. Thus, facilitating the conversion between local, Euclidean coordinates (X, Y and Z) and global, geographic coordinates (longitude, latitude and height/altitude).

But there is also the orientation part...

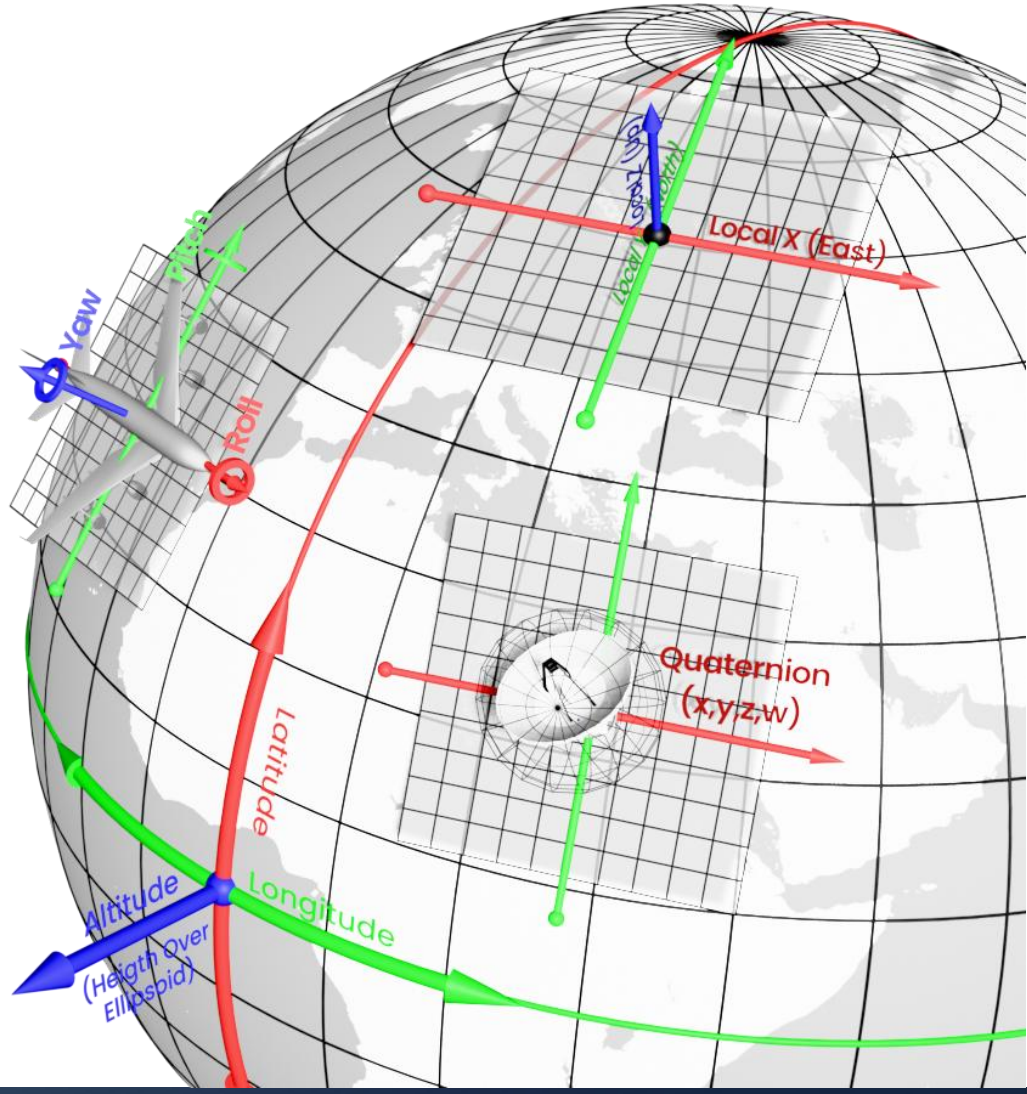


OGC GeoPose in a Nutshell

There are several implementation targets in the standard, but the “basic” GeoPose types are rather simple:

```
var basicAnglesGeoPose = {
  “position”: {       // Geographic position based on WGS84
    “lat”: number,    // Latitude, in degrees [-90, 90].
    “lon”: number,    // Longitude, in degrees [-180, 180].
    “h”: number       // Height/altitude over WGS84 ellipsoid, in meters
  },
  “angles”: {         // Euler angles, but as “aircraft principal axes”
    “yaw”: number,    // Rotation around the vertical axis, in degrees
    “pitch”: number,  // Rotation around the lateral axis, in degrees
    “roll”: number    // Rotation around the transversal axis, in degrees
  }
}

var basicQuaternionGeoPose = {
  “position”: { /* Same as before */ },
  “quaternion”: {     // Unit quaternion for spatial rotation
    “x”: number,      // First imaginary component of the quaternion (i)
    “y”: number,      // Second imaginary component of the quaternion (j)
    “z”: number,      // Third imaginary component of the quaternion (k)
    “w”: number       // Real component of the quaternion
  }
}
```

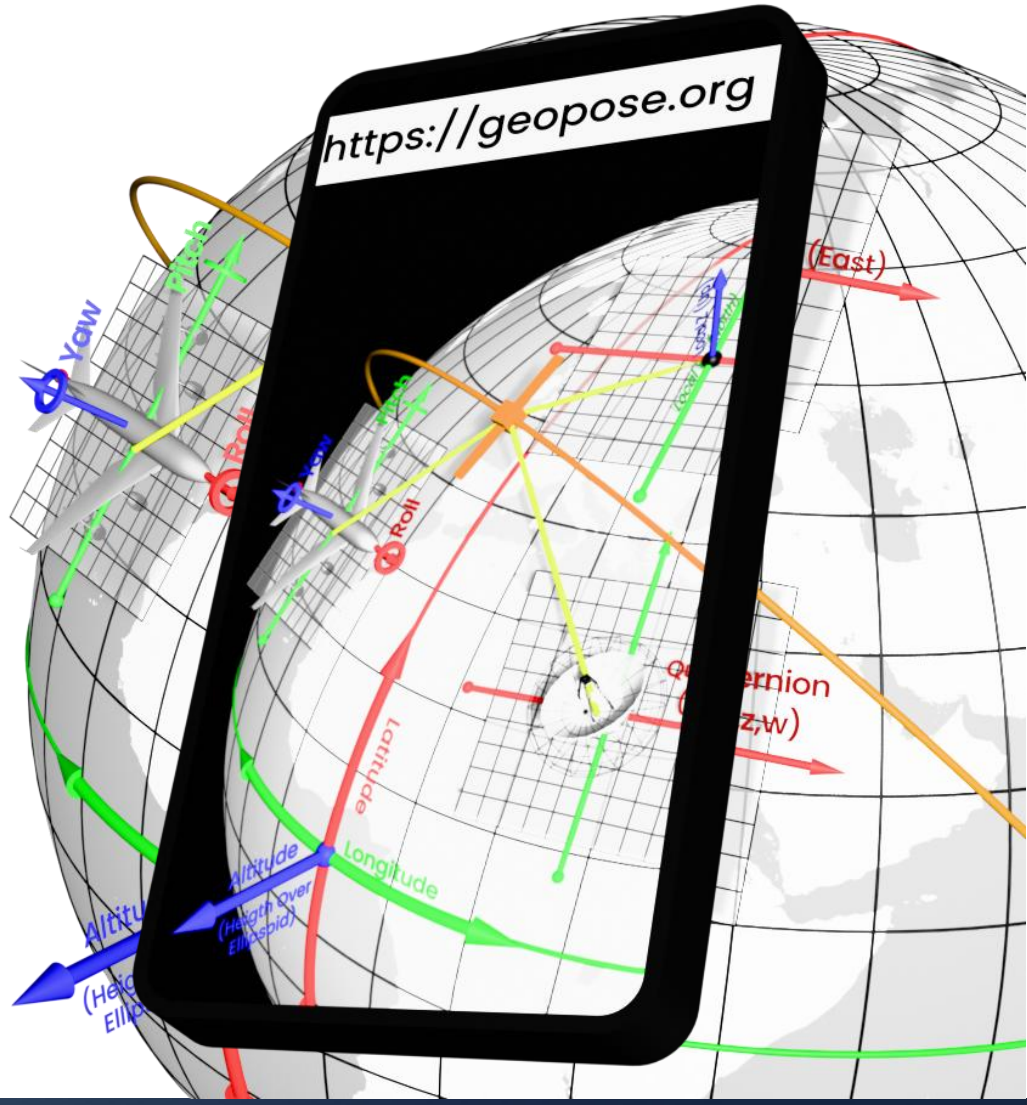


Ubiquitous Computing with OGC GeoPose

Ultimately, working with GeoPoses enables **planet-wide, shared, user interaction spaces** where billions of agents (including robotic systems) can communicate in an inter-operable way to achieve common goals.

The most direct application of this system is the creation of Digital Twins for large areas... but then you realize that there is no need to limit ourselves with artificial barriers.

With current mobile and XR technologies it is possible to create **GeoPose-based Digital Ecosystems using Discovery Services that automatically load and unload contents and experiences, as the users explore the world.**



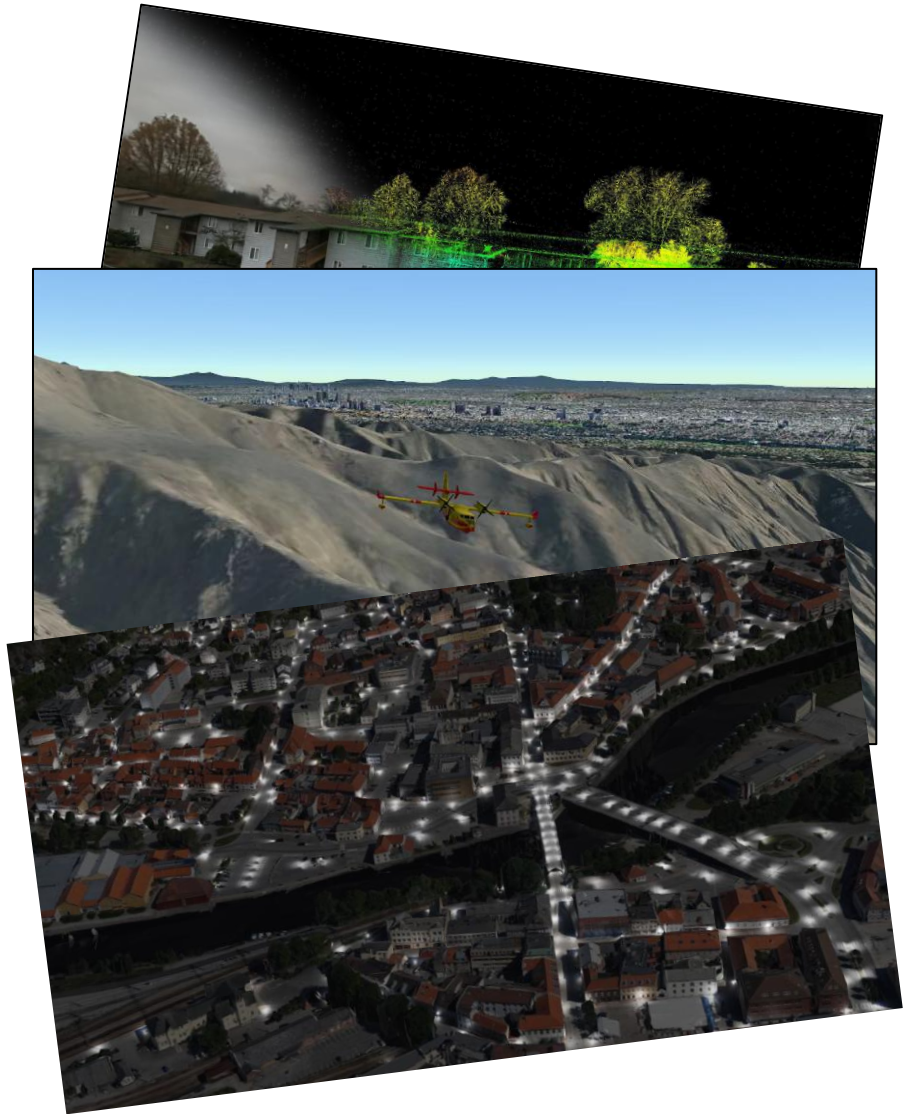
Benefits of using OGC GeoPose

Some interesting benefits of this approach are:

1. Multiple Visual representations: It doesn't really matter if they are 3D meshes, point clouds, gaussian splats.... they can all coexist in the same geographic interaction space (and often, in the very same view).

2. Pose Streams: By providing an array of GeoPoses with a temporal component, it is possible to create animations through simple interpolation.

3. Instantiation: Many objects in the world (signs, streetlights, cars, trees, etc.) are similar enough that we can be "copy-pasted" just with a new GeoPose.



Challenges Yet to Overcome

Nevertheless, there are several issues that have yet to be solved:

1. **Low Accuracy:** Currently, built-in sensor have rather poor accuracy (GPS only offers accuracy within 1-5 meters under open sky, and indoor location systems are still under development). To achieve the millimetric accuracy would require the use of extra devices or external services like Visual Positioning Systems (VPSs).

2. **Cybersecurity Risks:** Sharing your physical position with third parties poses significant privacy and security risks. And sharing photos with VPS system might also expose others to similar risks.

3. **Additional Computation:** Translating geospatial coordinates into the transform matrices that 3D hardware operates with requires the use of trigonometric functions that are computationally expensive.

Implementing (Basic-YPR) GeoPose

To show how much computation is required to work with OGC GeoPose, here is a small JavaScript implementation of the “Basic YawPitchRoll Angles” target:

```
// A Basic Euler Geopose is a simple JSON structure
let geopose = {
    "position": {"lat": 47.7, "lon": -122.3, "h": 11.5},
    "angles": {"yaw": 40, "pitch": -3, "roll": 0.0}
}

// Grab the values of the World Geodetic System (WGS84) ellipsoid
// @see https://en.wikipedia.org/wiki/World_Geodetic_System#WGS84
const equatorialRadius = 6378137.0;
const polarRadius = 6356752.314245;
const flatteningFactor = equatorialRadius / polarRadius;


// Use trigonometry to calculate the position of the
// space on the WGS84 Ellipsoid
let lng = geopose.position.lng  * (Math.PI/180),
    lat = geopose.position.lat * (Math.PI/180),
    alt = geopose.position.h,
    radius = equatorRadius + alt,
    lngSin = Math.sin(lng), lngCos = Math.cos(lng),
    latSin = Math.sin(lat), latCos = Math.cos(lat),
    geoX = lngCos * latCos,      geoY = latSin,
    geoZ = lngSin * latCos;

let position = new Vector(geoX * radius,
    geoY * radius * flatteningFactor, geoZ * radius);
```

```
// It is recommended to create the vertical vector for the camera
// lookat() function, and, once negated, for the gravity vector
let verticalVector = new Vector(geoX, geoY, geoZ);
verticalVector.normalize();
let gravity = verticalvector.clone().negate();


// Then, we can use a small hack to calculate the initial
// rotation of the entity/space in Euler angles
let x0 = latSin * equatorialRadius ,
    x1 = latSin * (equatorialRadius  + 1);
    y0 = latCos * polaRadius;
    y1 = latCos * (equatorialRadius  + 1) * equatorialRadius;
    dx = x1 - x0, dy = y1 - y0;
    l = Math.sqrt((dx * dx) + (dy * dy));

let rotation = new Euler(-Math.PI/2 + Math.acos(dx / l),
           Math.PI/2 - lng, 0, ‘YZX’);


// Finally, add the actual rotation of the GeoPose (make sure
you are applying the angular values to the right axes)
let rotation2 = new Euler(geopose.angles.roll,
    geopose.angles.yaw, geopose.angles.pitch, ‘YZX’);
```

Links

For more information, visit our website:
https://www.geopose.org

For the specification and documentation:
https://github.com/opengeospatial/GeoPose

For a developer sandbox (Typescript implementation):
https://github.com/opengeospatial/GeoPoseSandbox

For direct questions: (Mikel Salazar, Ph.D.)
contact@mikelsalazar.com

UBICOMP ISWC2025

14-16 Oct. 2025
Helsinki, Finland

Mikel Salazar, Ph.D.
VR Department. IFE
Halden, Norway
contact@mikelsalazar.com

OPEN AR CLOUD